Patterns with CUDA Programming

Stencil

- special case of map
  - 1D or multiple dimensions

- has regular data access pattern
  - each output depends on a neighborhood of inputs (block)
  - inputs independent of values relative to the output
  - can be implemented as
    - set of random reads for each output
    - shifts
- applications
  - two-grid and signal processing (convolution)
  - physics, mechanical engineering, CFD (PDE solvers over regular grids)
  - cellular automata
- different neighborhoods
  - square compact, ... sparse
  - cache optimizations
  - stencils issue samples required for neighboring elements
- boundaries of grids given to a processor (red dashed lines)
  - exchange data with other processors
  - additional communication costs

Implementation with Shift Operation

- beneficial for 1D stencils
- allow vectorization of data reads
- does not reduce memory traffic

Implementation with tiles

- multidimensional stencils
- shared memory (optimized for cache)
- example
  - two-dimensional array organized in row-by-row fashion
  - neighbors are stored in the same cache line, vertical far away
  - horizontal split
    - whole line does not fit cache, a lot of cache misses when accessing adjacent elements
  - vertical split
    - processors redundantly read the same cache line
  - strips (vertical)
    - each processor gets its strip of width equal to a multiple of cache line size
    - all processing goes sequentially from top to bottom to maximize cache reuse
    - multiple of cache line size prevents false sharing between adjacent strips on output

Communication

- continually the output of stencil is used as the input for the next iteration
  - double buffering
  - pointers to buffers are interchanged between iterations
- ping-pong communication
- boundary regions (halo) of the grid may need explicit communication with neighboring processors
  - halo size depends on stencil iteration
  - data exchange can take place on each B-th iteration when halo radius is increased, and some redundant computation takes place on each processor
  - latency hiding (update of internal grid cells) when waiting for halo exchange

Example: Heat distribution

- square surface, three edges touch boiling water; one edge is put in ice
- How is the heat distributed inside the surface?
- Laplace equation

$$\frac{\partial^2 T(x,y)}{\partial x^2} + \frac{\partial^2 T(x,y)}{\partial y^2} = 0$$

- discretized Laplace equation is in proper form for iterative solving

$$T(x,y) = \frac{1}{4}\cdot(T(x-h,y) + T(x+h,y) + T(x,y-h) + T(x,y+h))$$

- surface size $w \times h$ includes boundary values, tiles of size w BLOCK_SIZE+1 include halo

- result

- relations
  - hotSpot: CPU reference code
  - hotSpot.cu: stencil computation directly from the global memory
  - hotSpot GPU code, using local memory, alternate allocation
    - first threads copy data to the local memory
    - the kernel function calculates the values for which includes halo needed by threads on edges to correctly compute the next iteration
    - each thread transfers the value for which it is responsible
    - the host function invokes the kernel function BLOCK_SIZE times
    - threads can start with computation only when all data is in local memory
    - __syncthreads() represents a barrier in CUDA C
  - hotSpot.cu: kernel function implementation with shared memory
  - hotSpot GPU code, using local memory, dynamic allocation
    - threads in a warp or not accessing neighboring memory locations
    - degraded performance

Reduce

- a collective operation
- reduce pattern allows data to be combined
  - combiner function $f(a, b) = a \oplus b$
  - pairwise operation
  - associative operation
    - $(a \oplus c) \oplus (b \oplus c)$
    - operands can be combined in any order
    - floating point addition and multiplication are only partially associative due to the limited precision
  - commutative
    - $a \oplus b = b \oplus a$
    - not required, but enables additional reorderings
  - identity element
    - initial value of reduction
- serial and parallel patterns

Tiling

- use serial algorithm where possible
- do tree-like reduction to reduce communication costs

- process
  - break the work to tiles
  - operate on tiles separately
  - combine partial results from tiles
- serial and tree algorithm
  - performance cannot be worse than GPU
  - serial algorithm requires less storage for intermediate results

Theoretical Considerations

- sequential reduction of N operands
  - $N - 1$ reductions
  - each invocation of reduce function needs $\frac{1}{N}$ to complete
  - total execution time is $T_1 = \frac{N}{N} \cdot (N-1)$
- parallel tree-like reduction, $n = 2^k, k \in \mathbb{N}$
  - establishing communication takes A units of time
  - $N/2$ reductions in the first stage can go in parallel, $N/4$ in the second stage can go in parallel, ... 2 reduction in the last stage
  - altogether we have $\log_2 N$ stages with total $N - 1$ reductions
  - total execution time equals $T_p = (\frac{1}{2} + \frac{1}{4})\log_2 N$

Fusing Map and Reduce

- when map is feeding outputs directly into a reduction, the combination can be implemented more efficiently
- no need for synchronization between map and reduce stages
- no need to write intermediate results to memory or file
- map and reduce must be lifted in a same way

Example: Dot Product

- vectors **a** and **b** of length N
- dot product

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=0}^{N-1} a_i \cdot b_i$$

- relations
  - dotprod.cpu: CPU reference code
  - dotprod.cu: element-wise multiplication is performed on GPU; summation on CPU
  - dotprod.cu: summation split between GPU and CPU
    - less data to transfer back to CPU
    - local memory to store products
    - serial summation of products from local memory inside a block thread

- dotprod.cu: tree-like summation of values inside a block, stride is increasing
  - in each stage the number of working threads inside a warp decreases

- dotprod.cu: better warp management
  - tree-like summation of values inside a block, stride is decreasing
  - all threads in a warp are working (up to the last warp)
  - at each stage the number of warps halves
  - considerably improved performance

- dotprod.cu: for threads inside the final warp we use light-weight barrier with __syncwarp()
- dotprod.cu: generalization of solution to support block sizes not power of 2
- dotprod.cu: trick to compute largest power of 2 smaller or equal to the block size
- dotprod.cu: atomic add, no summation on CPU
- dotprod.cu: dynamic allocation of local memory
- dotprod.cu: managed memory solution

Scan

- also prefix scan
- produces all partial reductions of an input sequence
- associativity must hold as with reduction

- exclusive and inclusive scan
- operation
  - input sequence: $[a_0, a_1, a_2, ..., a_i, N-1]$
  - output
    - exclusive scan: $[I, a_0, a_0 \oplus a_1, a_0 \oplus a_1 \oplus a_2, ..., a_0 \oplus \cdots \oplus a_{i-1}]$
    - inclusive scan: $[a_0, a_0 \oplus a_1, a_0 \oplus a_1 \oplus a_2, ..., a_0 \oplus \cdots \oplus a_{N-1}]$

- sequential approach
  - loop-carried dependence
- parallel approach
  - loop-carried dependence
  - work inefficient
  - two sublistsm
  - count on associativity of the combiner function (2)
- combining scan
  - map
    - third scan map can be applied before the first stage and/or after the last stage
    - reduces data transfers
  - tree
    - similar scheme, can fit both with little extra work

Parallel scan with double buffering

- Hillis and Steele, 1986
- requires two buffers of length N

Two-phase parallel scan

- Blelloch, 1990
- tree-like approach
- requires only one buffer of length N
- work inefficient approach
- first phase: up-sweep
  - stride increases $0, ..., \lfloor \log_2 N \rfloor - 1$
  - operations are performed in-place and in computation in figure below
- second phase: down-sweep
  - stride decreases $\lfloor \log_2 N \rfloor - 1, ..., 0$
  - we displacements of elements used in computations in figure below

Tiled scan

- if problem cannot fit to one GPU, we split the problem to tiles
- tiled algorithm
  - each processor handles one tile (serial or parallel solution)
  - scan on last elements in each tile (blue) gives offsets (green)
  - application of offsets can be again performed inside each tile to get final values (gray)

GPU implementations

- tiled scan approach where size of a tile equals block size
- kiscan in GPU global memory holds the value of last element in each tile
- kiadd uses it as a result of the scan kernel and is used by the additional to compute offsets
- local memory
  - shared, length array presents input and output buffer (tile)
- solutions for inclusive scan
  - scan.cu: serial scan inside blocks (tiles) using local memory, serial computation of offsets
  - scan2.cu: parallel double-buffering scan
    - double buffering inside a thread block (tile)
    - offsets are computed by redoes operation as each thread block has to do it on its own
    - addition of offsets goes in parallel
  - scan3.cu: work efficient scan
    - only the scanformat is updated